

Convolutional Neural Network (CNN)

Why CNN?

CNN do 3 things to make image classification practical.

- 1.) It reduces the number of inputs
- 2.) Tolerate small shift in where the pixels are in the image
- 3.) Take advantage of correlations that we observe in complex images.

Steps Involved in CNN:

(i) Apply Filter (kernel) on the input image.

Filter is generally square matrix 3 pixels by 3 pixels and the intensity of each pixel is determined by backpropagation.

Before Training, we start with random pixel values in filter. So After training with backpropagation

we end up with something more useful.

To apply filter to the input image, we overlay filter into the input image and multiply

together with each overlapping pixel. And add each product with final value. (Dot Product)

"By computing the Dot Product of input & Filter, we can say that filter is convolved with the input and that's what it gets convoluted Neural Network name.

(ii) Apply bias to final value and store it in feature Map.

(iii) We shift the filter by 1 pixel (or more) and compute final value & then add bias and put it in feature Map.

(iv) Run the feature Map thru Activation function (ReLU) and means all of the negative values set to zero.

(v) No, we apply another filter but we just the 'maximum' in the filter. (Max Pooling)

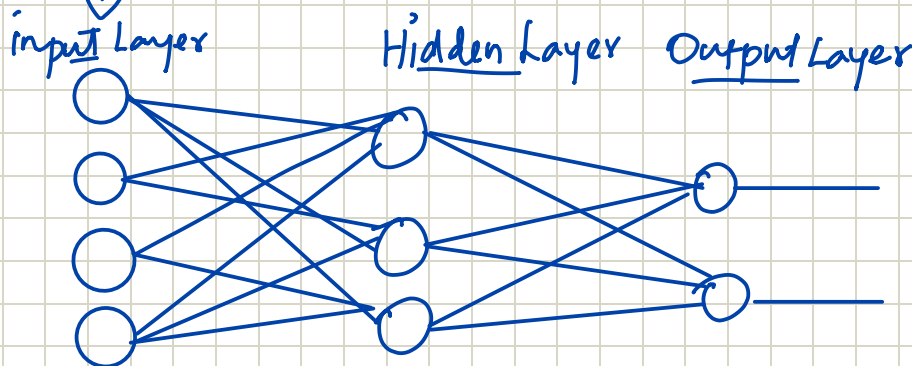
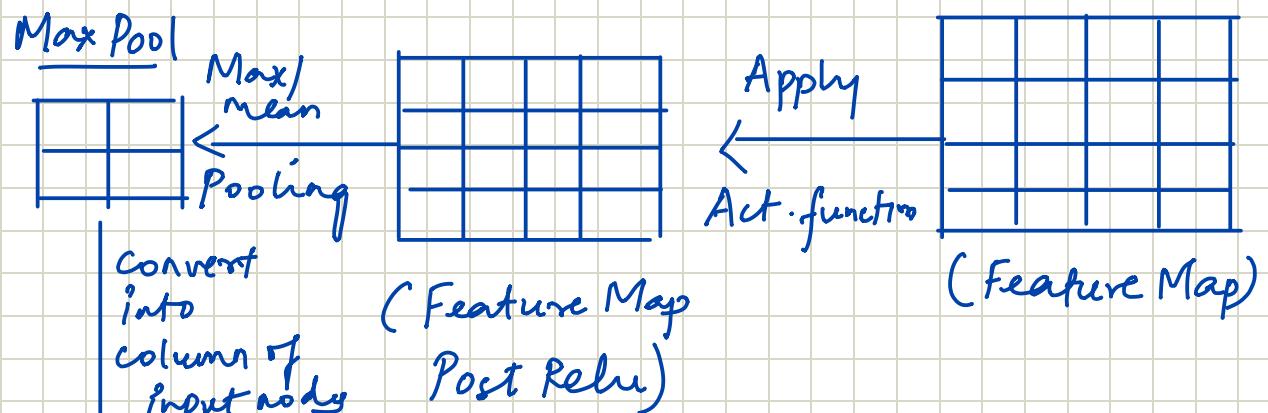
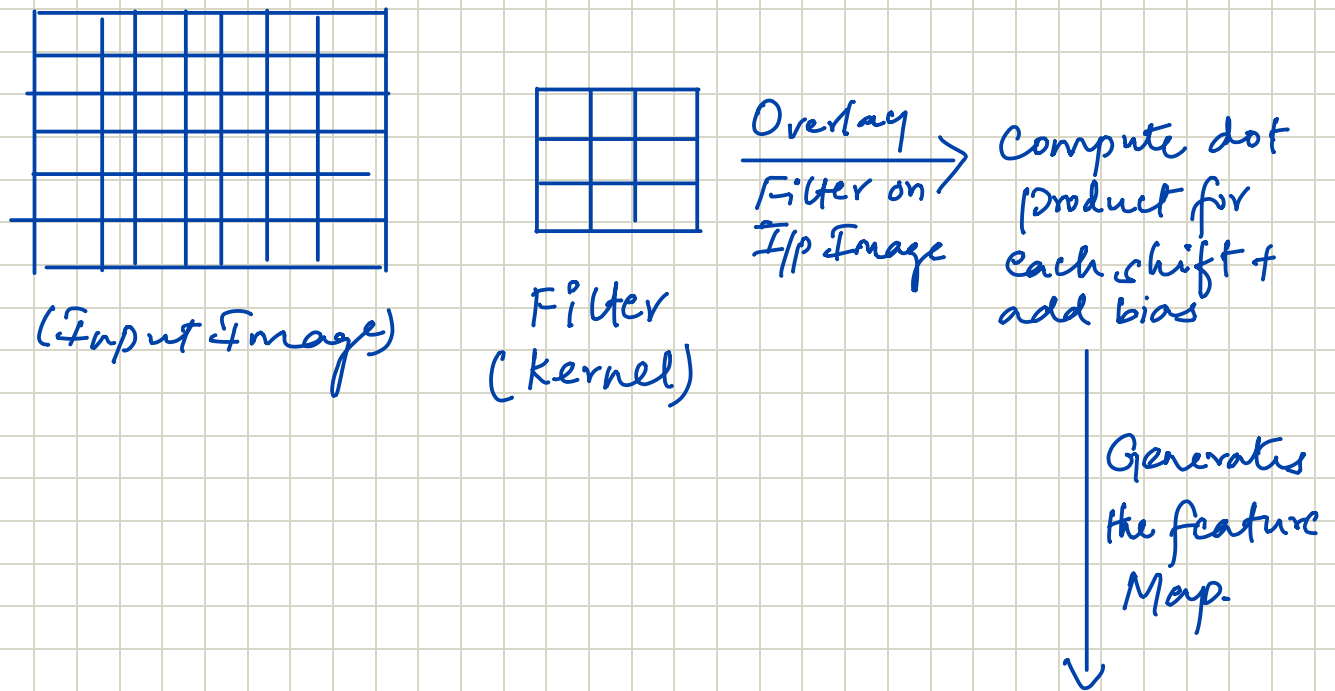
This max pooling represents the exact match of the filter on input image.

(Alternatively we can calculate the average value for each region and that would be called as "average or mean pooling".

(vi) Now The pooled layer converted into column of input nodes.

(vii) Plug the input nodes to normal neural network we use regularly.

Summary:



Importance of CNN:

- (i) weight sharing & spatial coherence: (Filter/kernel) reduces parameters $\longrightarrow$ So faster computation & lesser memory usage.
Spatial coherence $\longrightarrow$ robust to variations (small)
- (ii) Equivariance $\longrightarrow$ predictable output changes
(reliable to input transformations)
- (iii) Memory Efficient.

Padding:

When the filter applied on input image, the filter convolves. The pixels in the middle are used more often than edge/corner pixels. The information on the border is not preserved as well the information in the middle.

Padding is a simple technique where we add 1 layer of zeroes around the input image and prevents the shrinkage of the input image.

p - num of layers of padding added. So $(n \times n)$ image with $(f \times f)$, we get $(n + 2p \times n + 2p)$ (Image After Padding)

$(n + 2p) \times (n + 2p) * (f \times f) \longrightarrow (n + 2p + f - 1) \times (n + 2p + f - 1)$ Output Image.

2 Types of Padding

Valid Padding
(It implies no padding)

$$(n \times n) * (f \times f) \rightarrow (n \times f - 1) \times (n \times f - 1)$$

Same Padding
(We had p layer to maintain the same dimension of input image)

$$(n + 2p \times n + 2p) * (f \times f) \rightarrow (n \times n) \text{ Image.}$$

$$p = \frac{(f - 1)}{2}$$

Key Components of Convolution Layers

- (i) Filters (Kernels)
- (ii) Stride (Stepsize of kernel)
Larger strides give smaller output feature maps & faster computation.
- (iii) Padding
- (iv) Activation function.

Types of Convolution Layers:

- (i) 2D Convolution
- (ii) Depthwise separable Convolution
- (iii) Dilated (Atrous) Convolution

Convolved Layers

vs

Fully Connected Layers

- * Connectivity is Local (neuron to local region)
- * Lower parameters
- * spatial information is preserved
- * feature Extraction

- * Global Connectivity (each neuron to global)
- * Higher parameters
- * spatial information is lost
- * Classification, Regression

Introduction to Pooling Layers:

Pooling Layers in CNN is to reduce the spatial dimension of the feature maps while retaining the most important information.

$$\left(\frac{n_h - f + 1}{s} \right) \times \left(\frac{n_w - f + 1}{s} \right) \times n_c$$

h - height
w - width
c - channels from feature map

Types:

- 1.) Max Pooling
- 2.) Average/Mean Pooling
- 3.) Global Max/Arg Pooling

Rule of Thumb: (2×2) filter & stride $(2, 2)$ for Pooling.

Pros:

- * Dimensionality Reduction
- * Translation Invariance
- * Feature Selection

Max Pooling $\rightarrow$ most salient features

Average Pooling $\rightarrow$ balanced representation.

Cons:

- * Information Loss: pooling reduces spatial resolution
(some loss of important fine details)
- * Hyperparameter tuning: filter & stride size.
- * Over-smoothing: may blur the crucial features.

Fully Connected Layers (dense): each neuron in one layer will interconnect with all neurons in subsequent layers.

Convolution Layers $\xrightarrow[\text{Layers}]{\text{Pooling}}$ Fully Connected Layers

Forward Pass
(Convolution Layer +
Activation function +
Pooling Layers)

Loss
 $\Rightarrow$ Calculation $\Rightarrow$

Backward Pass
(backpropagation)
(weights & bias
of all layers is
adjusted during
training)